Four pillars of OO

Presented by: Sander Huijsen
Adrian Gould
Murray Hay

ICT40120 – Cert IV of Information Technology
Cluster IOT

Unit code	Unit title
ICTPRG430	Apply introductory object-oriented language skills
ICTPRG444	Analyse software requirements

Session 6

FOUR PILLARS OF OO

OO: Four Pillars

- Object-oriented programming is said to rest on 4 pillars:

  1. Abstraction
  2. Polymorphism
  3. Inheritance
  4. Encapsulation

- Easy to remember: A PIE
  - Raspberry Pi(e)?

(A) Abstraction

- Abstraction is the process of removing certain attributes to focus attention on details of greater importance.
- Process of abstraction: modelling
- Example: Person
  - There are about 7 billion unique people
  - Yet everyone can “picture” the abstract “Person”

Background created by Adrian Gould

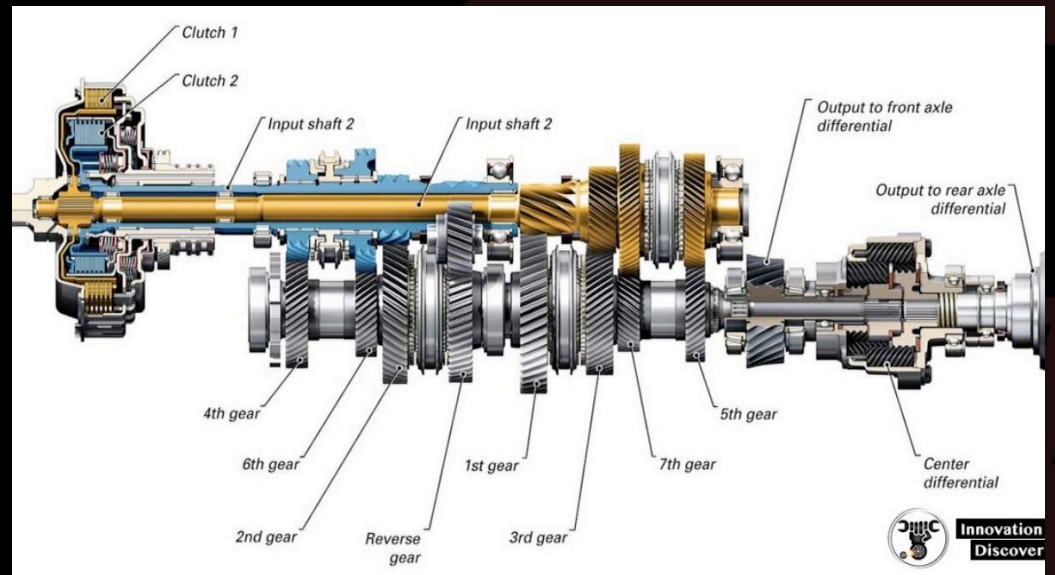
Classes

- Classes in OO are, by definition, abstractions
- They *model* “real life” objects, but never fully
- In a Student Management System, you may model students as object with certain attributes:
  - Yes: name, address, date of birth
  - No: hair colour, shoe size, height, etc.

Background created by Adrian Gould

Abstract operations

- You can also create abstractions for *operations*:
  - "Make tea" is an abstraction of an entire process:
    - *put on the kettle, get a cup, pour the water, put in tea, wait, etc…*
  - Automatic shift is an abstraction of the intricate mechanics
    - Park, Drive, Neutral, Reverse is all you need
    - Drivers of early cars had it much tougher
  - Database operations on an object-level in stead of writing SQL



https://innovationdiscoveries.space/what-are-the-main-components-of-the-gear-box/

Abstraction vs generalisation

- The most abstract idea is also the most generalized
- E.g., a “Person” can be anyone person (highly abstract)
- “Child” is more specific and slightly less abstract (but still a person)
- On the other end of the spectrum: a Person is a “Living Being” (more abstract)

Abstraction in Python

- An abstract person in Python: a contact in your *phonebook*

```
class Contact:
    def __init__(self, first_name, last_name, phone_number):
        self.first_name = first_name
        self.last_name = last_name
        self.phone_number = phone_number
```

(I) Inheritance

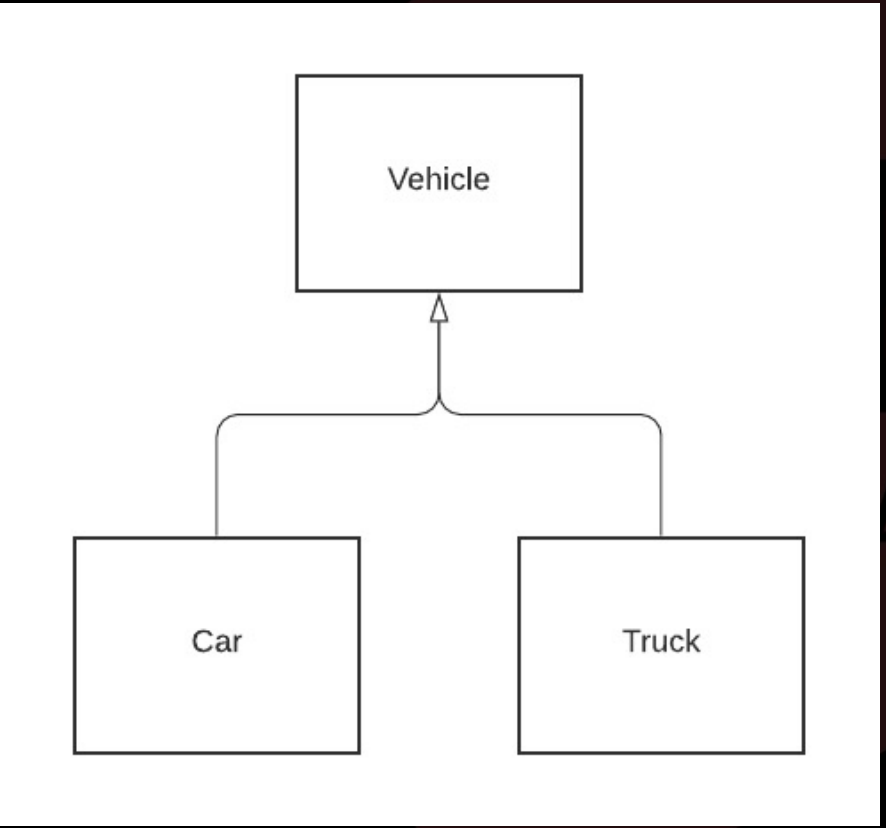
- Inheritance is the process of basing objects (classes) on other objects (classes)
  - The so-called “subclass” is, by definition, less general than the class it derives from (called the “super class”)
  - Subclasses get ***all*** attributes and functionality of the super class

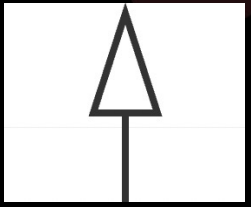
Not the inheritance we mean here:

https://falonilaw.com/talking-to-kids-about-inheritance/

Inheritance

- Consider the concept of Vehicle
  - A Car can be considered a subclass of Vehicle
  - A Truck too
- Inheritance is indicated with a special arrow that points toward the ***super*** class





Inheritance in Python

- In Python, inheritance is described by putting the superclass within parentheses (more about this later):
  - Assume a class **Vehicle** exists

```
class Car(Vehicle):
    # Attributes and methods follow here
    pass
```

(E) Encapsulation

- Encapsulation is:
  - Bundling of data with methods that operate on that data
  - Restricting direct access to some of an object's components

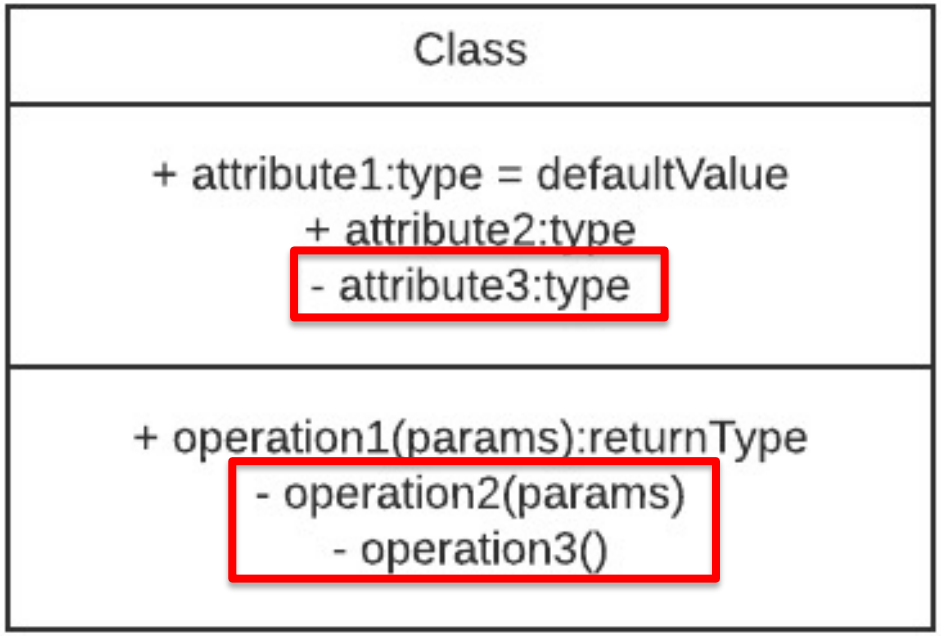
- In object-oriented programming:
  - Methods are functions (bundled in a class) that operate on the attributes of that class
  - Attributes are (obviously?) also bundled in the class

Encapsulation

- The class shows:
  - Attributes
  - Operations (methods)

- Attributes and methods can be:
  - *public*: not hidden at all (+)
  - *protected*: hidden from all but subclasses (#)
  - *private*: completely hidden (-)

- Hidden attributes and methods are indicated with a red line



Encapsulation in Python

- Encapsulation in Python: a contact in your *calendar*

```
class Contact:
    def __init__(self, first_name, last_name, date_of_birth):
        self.first_name = first_name
        self.last_name = last_name
        self.date_of_birth = date_of_birth

    def age(self):
        """Calculate and return Contact's age based on DoB."""
        # Implementation goes here
```

(P) Polymorphism

- This literally means many (poly) forms (morph)

- There are different forms of polymorphism:
  - Based on a common interface
  - Using a name that denotes instances of different classes that are related by a common superclass
    - *Sounds a bit like inheritance, right?*
  - Using a single function/method that can handle values independent of their type
    - *Parametric polymorphism*

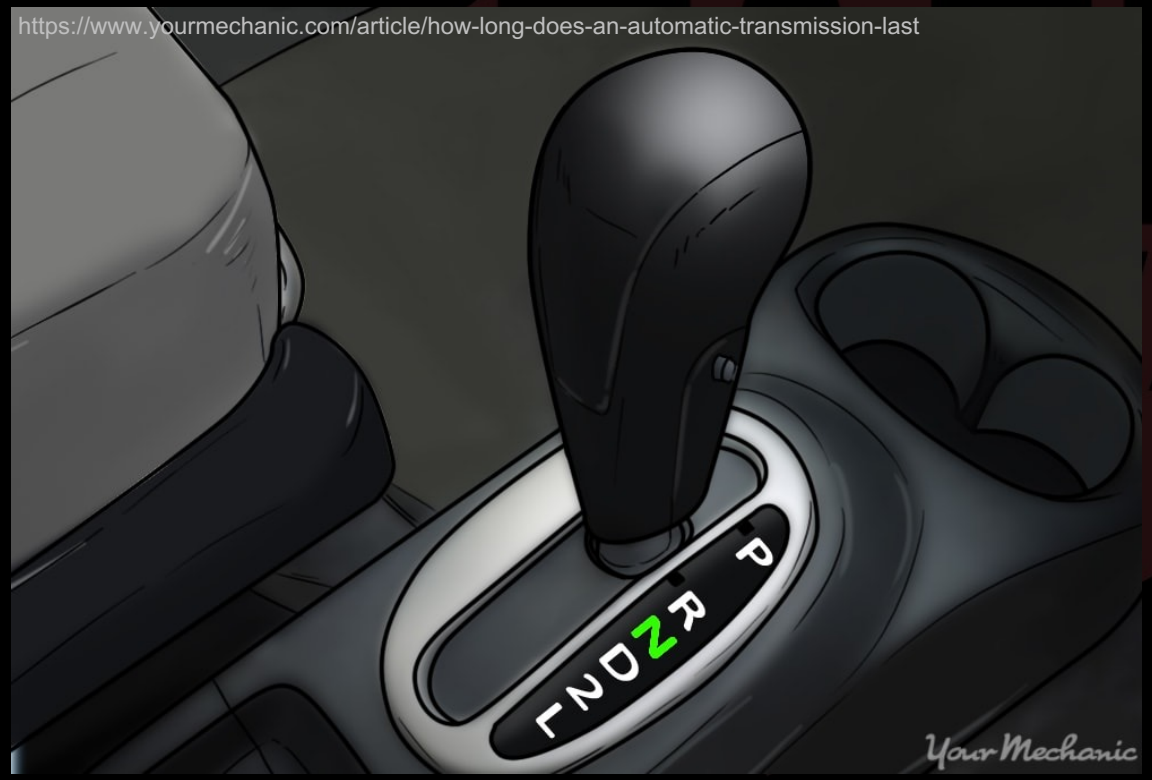
Background created by Adrian Goold

Polymorphism: interface

- Compare with automatic gearbox: if you know how to use it, you can drive ***any*** car that has an automatic gearbox

- The shift stick is the *interface* that you use

- The interface is said to be *implemented*



Polymorphism: subclass

- Pretend an object of type Subclass to be of type Superclass, then call a method of Superclass
- You don't have to know the actual type of the object and it will still execute the correct behaviour
- In a sense, the superclass already implemented the interface, and all subclasses can just use it
  - Subclasses can have their own implementation, though!

Polymorphism: parametric

- Python does not have templates or generics, so this type of polymorphism is less relevant
  - See Wikipedia for examples from C#/Java
- For example, you can sort any list of object without telling Python what the type is
  - But only if each object can be compared with other objects of the same type

Duck typing

- Remember: Python is a dynamically typed language
- Duck typing: if it walks like a duck and quacks like a duck, it probably is a duck.
- In other words: you don’t have to know what type it is to call a method (as long as the object has it)

Background created by Adrian Gould

Exercise: Pillars of OO

- Provide one of example of:
  - Abstraction
  - Polymorphism (of any form)
  - Inheritance
  - Encapsulation

  - *Think about real-life objects and situations*
  - *Don't use any already shown examples*
  - *You may do additional "research," but don't just copy-paste Wikipedia*

Background created by Adrian Gould

Session 6

END

Background created by Adrian Gosid